PSG COLLEGE OF TECHNOLOGY

Coimbatore, Tamil Nadu

DEPARTMENT OF INFORMATION TECHNOLOGY

23IF02 - SYSTEM AUTOMATION

A PROJECT REPORT ON

ROBOTIC ARM JOINT HEALTH MONITORING SYSTEM

A safety-gated predictive-maintenance and sensor-integrity platform using Python, rule-based NLP and machine learning

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted by

Lakshmandev Vk - 24i433

ABSTRACT

Industrial robotic arms operate for long periods under heavy load, and the failure of a single joint can halt a production line or endanger people nearby. Threshold alarms alone react too late, while purely data-driven predictions cannot be trusted to protect safety on their own. This project presents the Robotic Arm Joint Health Monitoring System, a modular Python pipeline that combines four independent evidence channels to produce exactly one definitive operational decision for every robotic arm: SAFE, MONITOR, SCHEDULE_MAINTENANCE or EMERGENCY_STOP.

A deterministic safety engine enforces joint-specific torque, temperature and vibration limits and handles missing or invalid data in a fail-safe manner. An anti-spoofing module verifies the integrity of the sensor stream by detecting frozen sensors, replayed timestamps, implausible rates of change and cross-sensor contradictions. A zero-API natural-language module reads free-text technician notes using regular expressions and categorised dictionaries, crediting documented repairs and ignoring operational chatter. A StandardScaler and RandomForest pipeline, trained with class weighting on a chronological split to handle the 2.8% failure rate, estimates the 24-hour failure risk of each joint. A priority-based decision engine merges these channels so that a safety stop can never be overridden by a low predictive risk.

The system was evaluated on 2,160 synthetic historical records, a 36-row snapshot of six arms with six joints each, twelve technician notes and an eight-sample sensor-freezing attack. The model reached an accuracy of 98.84%, recall of 91.67% and ROC-AUC of 0.997 on a 432-sample hold-out set, and all 22 automated tests passed. In simulation the system returned one SAFE, one MONITOR, one SCHEDULE_MAINTENANCE and three EMERGENCY_STOP decisions, including a frozen-sensor attack and a torque excursion that the model alone had rated as low risk.

Keywords: Predictive Maintenance, Robotic Arm, Condition Monitoring, Sensor Spoofing, Random Forest, Maintenance Log Analysis, Fail-Safe Design, Python.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Methodology 2

1.6 Organisation of the Report 3

CHAPTER 2 TECHNOLOGY OVERVIEW 4

2.1 Condition Monitoring and Predictive Maintenance 4

2.2 Deterministic Safety Gating 4

2.3 Sensor Spoofing and Integrity Attacks 4

2.4 Rule-Based Natural Language Processing of Maintenance Logs 5

2.5 Random Forest and Imbalanced Classification 5

2.6 Python Ecosystem and Tooling 5

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 6

3.1 Requirement Analysis 6

3.2 System Architecture 7

3.3 Data Flow 7

3.4 Decision Classes and Precedence 8

3.5 Datasets 8

3.6 Project Folder Structure 9

CHAPTER 4 IMPLEMENTATION 10

4.1 Dataset Generation 10

4.2 Configuration Loader 10

4.3 Data Validator 10

4.4 Deterministic Safety Engine 11

4.5 Sensor Spoofing Detection 11

4.6 Maintenance Log Analyzer 12

4.7 Predictive Wear Model 12

4.8 Decision Engine 13

4.9 Reporting and Command-Line Interface 13

CHAPTER 5 TESTING AND RESULTS 14

5.1 Test Environment 14

5.2 Automated Test Suite 14

5.3 Operational Results 15

5.4 Discussion 16

5.5 Limitations 17

CHAPTER 6 SECURITY CONSIDERATIONS 18

6.1 Threat Model 18

6.2 Design Trade-offs 18

6.3 Safety Disclaimer 18

CHAPTER 7 FUTURE ENHANCEMENTS 19

CHAPTER 8 CONCLUSION 20

REFERENCES 21

APPENDIX A OPERATIONAL COMMANDS 22

APPENDIX B OUTPUT ARTEFACTS 23

APPENDIX C SOURCE FILES AND TESTS 24

LIST OF FIGURES

Figure 3.1: Overall system architecture 7

Figure 4.1: Output of the dataset generator 10

Figure 4.2: Model training and evaluation report 12

Figure 5.1: Output of pytest -v: 22 tests passed 15

Figure 5.2: Operational summary produced in simulation mode 15

LIST OF TABLES

Table 2.1: Technology stack used in the project 5

Table 3.1: Hardware and software requirements 6

Table 3.2: Operational decisions in order of precedence 8

Table 3.3: Datasets used in the project 8

Table 4.1: Evaluation of the predictive model on the 432-sample hold-out set 12

Table 4.2: Confusion matrix of the hold-out set 13

Table 5.1: Summary of the automated tests 14

Table 5.2: Final operational decision for each robotic arm 16

Table 6.1: Threats, detection methods and system responses 18

Table B.1: Files written by the pipeline 23

Table C.1: Source modules 24

Table C.2: Documentation delivered with the project 24

CHAPTER 1
INTRODUCTION

1.1 Background

Robotic arms are among the most widely used machines in modern manufacturing. A six-joint arm performs welding, assembly, handling and inspection tasks with high repeatability, but every joint contains motors, gearboxes and bearings that wear with use. Typical symptoms of developing faults include rising vibration, increasing joint temperature and a growing torque demand caused by gearbox backlash or lubrication loss. If these symptoms are not noticed in time, the result is unplanned downtime, damaged workpieces and, in the worst case, a hazard to nearby operators.

Condition monitoring addresses this problem by streaming sensor readings from every joint and analysing them continuously. Two complementary techniques are used. Rule-based safety limits react immediately and predictably when a reading crosses a dangerous value. Predictive models learn from historical data and estimate how likely a joint is to fail in the near future, which allows maintenance to be planned before a breakdown occurs. Technicians also record valuable observations as free text, such as a noisy gearbox or a replaced bearing, and this information is rarely used by automated systems.

A further concern is the trustworthiness of the data itself. Industrial networks can deliver stale, frozen or deliberately falsified readings, and a monitoring system that blindly trusts its inputs can be made to report a healthy machine while the real joint is failing. This project combines all of these ideas into one pipeline that treats safety, data integrity, human knowledge and prediction as separate evidence channels and merges them under a strict priority rule.

1.2 Problem Statement

Existing simple monitors either rely only on fixed thresholds, which cannot anticipate failures, or only on machine-learning scores, which are not suitable as a safety mechanism and can be misled by corrupted data. There is a need for an open, transparent and testable system that can:

- stop an arm immediately and deterministically when a physical safety limit is crossed, irrespective of any model output;
- verify that the sensor data can be trusted before relying on it, and detect frozen, replayed or implausible readings;

- extract maintenance-relevant knowledge from free-text technician notes without calling any external service;
- predict the 24-hour failure risk of every joint on a heavily imbalanced dataset; and
- reduce all of this evidence to exactly one clear, explainable decision per robotic arm.

1.3 Objectives

1 To design a configuration-driven pipeline in which all safety, torque, predictive and cyber thresholds are held in a single validated JSON file.

2 To implement a deterministic safety engine with a fail-safe response to missing, NaN or infinite telemetry.

3 To implement an anti-spoofing module that detects frozen sensors, replayed timestamps, implausible rates of change and cross-sensor contradictions.

4 To implement a rule-based log analyser that separates active faults, documented repairs and irrelevant chatter.

5 To train and evaluate a RandomForest wear-prediction model using a chronological, leakage-free split and class weighting.

6 To implement a decision engine with the precedence EMERGENCY_STOP, SCHEDULE_MAINTENANCE, MONITOR, SAFE and to export CSV, JSON and Markdown reports.

7 To verify every requirement with an automated pytest suite and demonstrate all four decisions in simulation.

1.4 Scope of the Project

The project covers the complete software pipeline, from data generation through validation, analysis, prediction and decision-making to reporting. Telemetry and failure labels are synthetic and were produced with empirical degradation equations, so every layer can be demonstrated and tested safely and repeatably. The system is a decision-support and diagnostic tool. It is not a certified industrial safety controller; real installations require independently certified hardware safety relays and dual-channel interlocks (ISO 13849 and IEC 61508). Hardware design, real-time control and deep-learning prediction are outside the scope.

1.5 Methodology

The system was developed in layers. Each module was written, unit-tested in isolation and then integrated with the previous modules.

1. **Requirement analysis:** the monitored parameters, the four permitted decisions and the safety rules were defined.
2. **Architecture design:** a modular package with one responsibility per file and a shared configuration was chosen (Chapter 3).
3. **Data generation:** reproducible historical, current, log and attack datasets were generated.
4. **Implementation:** validator, safety engine, spoof detector, log analyser, predictive model, decision engine and reporter were implemented (Chapter 4).
5. **Testing and validation:** 22 automated tests and a six-arm simulation verified the behaviour (Chapter 5).

1.6 Organisation of the Report

Chapter 1 introduces the project. Chapter 2 reviews the underlying concepts and tools. Chapter 3 presents the requirements, architecture and design. Chapter 4 describes the implementation of each module with supporting screenshots. Chapter 5 presents testing, results and limitations. Chapter 6 discusses security considerations. Chapter 7 proposes future enhancements and Chapter 8 concludes the report. References and appendices follow.

CHAPTER 2
TECHNOLOGY OVERVIEW

This chapter introduces the concepts and tools on which the project is built and explains why each was selected.

2.1 Condition Monitoring and Predictive Maintenance

Condition-based maintenance replaces fixed service intervals by decisions that depend on the measured state of the machine [4]. Sensors on each joint report quantities such as torque, temperature and vibration. Increasing vibration and temperature are classic indicators of bearing and gearbox wear, while a rising torque demand points to friction or backlash. Predictive maintenance goes one step further and uses historical data to estimate the probability that a component will fail within a given horizon. In this project the horizon is 24 hours.

2.2 Deterministic Safety Gating

A safety gate is a rule that is evaluated directly on the measured values and that always produces the same output for the same input. Because it contains no learned component, its behaviour can be inspected, tested and justified. In this system the safety engine runs first and independently, and its EMERGENCY_STOP result has the highest priority in the decision engine. A statistical model, however accurate, is never allowed to cancel a safety stop. Invalid inputs such as NaN, infinite or missing values are also treated as unsafe, which is the fail-safe principle: when the state is unknown, the machine is stopped.

2.3 Sensor Spoofing and Integrity Attacks

A monitoring system is only as reliable as its data. Three simple but realistic attack or fault patterns are considered. A **frozen sensor** repeats the same value, so the standard deviation of a recent window becomes zero while the real machine continues to change. A **replay** resends old messages, which shows up as duplicate or non-advancing timestamps. A **step jump** changes by an amount that is physically impossible between two consecutive samples, which is detected by a rate-of-change limit. A cross-sensor check adds further evidence: if one sensor claims a calm joint while the others show heavy load, the readings contradict each other.

2.4 Rule-Based Natural Language Processing of Maintenance Logs

Technician notes are short, informal sentences. Instead of calling an external language-model service, this project uses regular expressions and categorised keyword dictionaries. Notes are matched to a joint and an arm, classified as a fault indication (for example gearbox backlash or abnormal noise), a repair or replacement, or irrelevant chatter, and converted into a concern score between 0 and 1. A documented repair reduces the earlier concern, which prevents an old fault from raising alarms forever. The approach is transparent, fast, free of network dependencies and fully testable.

2.5 Random Forest and Imbalanced Classification

A random forest is an ensemble of decision trees trained on random subsets of the data and features, whose votes are averaged [2]. It handles non-linear relationships, needs little tuning and gives probability estimates, which are used here as the failure risk. Failures are rare in the data (2.8% in the test set), so a model that always predicts no failure would already be 97% accurate. To avoid this trap the classes are weighted as balanced, the evaluation focuses on precision, recall, F1-score and ROC-AUC, and the confusion matrix is reported [5]. The data are split chronologically so that the model is tested only on the future of its training period.

2.6 Python Ecosystem and Tooling

The project uses Python with NumPy and pandas for data handling, scikit-learn for the model and pre-processing [3], joblib for model persistence, pytest for testing and Git for version control. Python was chosen for its mature data-science libraries and because the same code runs unchanged on Windows, Linux and macOS.

Table 2.1: Technology stack used in the project

Component	Technology	Purpose
Language	Python 3.13	Implementation of all modules
Data handling	NumPy, pandas	Telemetry tables, windows and statistics
Machine learning	scikit-learn (StandardScaler, RandomForestClassifier)	Wear prediction pipeline
Persistence	joblib	Serialised model artefact
Log analysis	Python re module and keyword dictionaries	Zero-API NLP of technician notes
Configuration	JSON with strict schema validation	Centralised thresholds
Testing	pytest 8.3.3	22 automated tests
Version control	Git	Source management

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

- The system shall load and validate telemetry for six arms with six joints each.
- The system shall stop an arm immediately when torque, temperature or vibration crosses its critical limit.
- The system shall treat missing, NaN or infinite readings as a fail-safe condition.
- The system shall classify the integrity of the sensor stream as TRUSTED, SUSPICIOUS or CRITICAL_INTEGRITY_FAILURE.
- The system shall convert technician notes into a per-arm maintenance concern score and credit documented repairs.
- The system shall predict the 24-hour failure risk of every joint.
- The system shall output exactly one decision per arm using the fixed precedence.
- The system shall export results as CSV, JSON and Markdown and keep an audit log.

3.1.2 Non-Functional Requirements

- **Determinism:** safety decisions must not depend on any learned model.
- **Reproducibility:** data generation and training must give the same results for the same seed.
- **Configurability:** all thresholds must be editable without changing code, and invalid configuration must be rejected.
- **Modularity:** each subsystem must be testable on its own.
- **Explainability:** every decision must carry the reason that triggered it.

3.1.3 Hardware and Software Requirements

Table 3.1: Hardware and software requirements

Item	Requirement
Processor	Any modern 64-bit processor
Memory	4 GB RAM or more
Operating system	Windows 10/11, Linux or macOS (tested on Windows)
Software	Python 3.13, pip, Git, command prompt or PowerShell
Python packages	numpy, pandas, scikit-learn, joblib, pytest (see requirements.txt)

3.2 System Architecture

The system follows a layered pipeline architecture, shown in Figure 3.1. Raw inputs are first validated. The validated data are then examined by four analysis channels that do not depend on one another: the safety engine, the spoof detector, the maintenance log analyser and the predictive model. The decision engine collects their findings and applies the priority rule, and the reporting module writes the results.

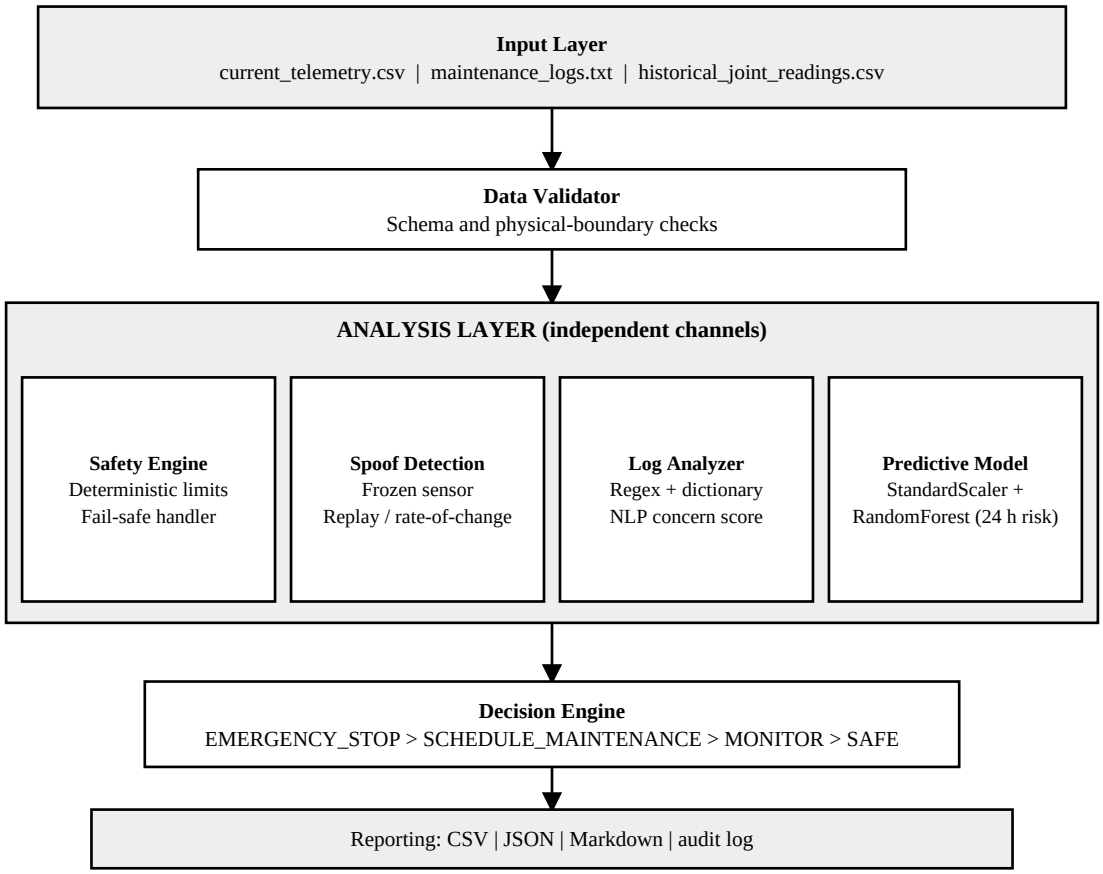


Figure 3.1: Overall system architecture

The independence of the four channels is a deliberate design choice. A failure or manipulation of one channel, for example a spoofed vibration signal that fools the model, does not disable the others. In particular the safety engine never reads the model output, so a safety stop is always based on measured physical values.

3.3 Data Flow

1 The orchestrator (main.py) loads the configuration and checks it against the schema.

2 Telemetry is read from CSV and validated for columns, types and physical boundaries.

3 The safety engine evaluates each joint against its critical limits and returns a fail-safe result for invalid data.

4 The spoof detector examines the sequence of readings of each joint and assigns an integrity status.

5 The log analyser parses the technician notes and returns a concern score for every arm.

6 The predictive model returns the 24-hour failure probability of every joint.

7 The decision engine aggregates all joints of an arm, applies the precedence and records the critical joint and the reason.

8 The reporter writes the CSV, JSON and Markdown outputs and the audit log.

3.4 Decision Classes and Precedence

Every arm receives one of four decisions. When several conditions apply to different joints of the same arm, the decision with the highest precedence wins.

Table 3.2: Operational decisions in order of precedence

Priority	Decision	Meaning
1 (highest)	EMERGENCY_STOP	A critical safety limit is crossed, data are invalid, or the sensor integrity has failed. The arm must be stopped.
2	SCHEDULE_MAINTENANCE	High predictive risk and/or severe maintenance concern. Maintenance must be planned.
3	MONITOR	Moderate wear or an active technician note. Keep the arm under observation.
4 (lowest)	SAFE	All joints are within nominal limits.

3.5 Datasets

Four datasets are produced by a single reproducible generator. They are listed in Table 3.3.

Table 3.3: Datasets used in the project

Dataset	File	Size	Purpose
Historical readings	data/historical_joint_readings.csv	2,160 rows (6 arms x 6 joints x 60 time steps)	Training and evaluation of the model
Current telemetry	data/current_telemetry.csv	36 rows (6 arms x 6 joints)	Snapshot analysed by the pipeline
Maintenance logs	data/maintenance_logs.txt	12 technician notes	Faults, repairs and chatter for the NLP module

Dataset	File	Size	Purpose
Spoofed sensor cases	data/spoofed_sensor_cases.csv	8 sequential samples	Frozen-sensor attack scenario

3.6 Project Folder Structure

```
Occ/
|-- config/thresholds.json        Safety, torque, predictive and cyber thresholds
|-- src/
|   |-- config_loader.py          Schema loading and validation
|   |-- data_validator.py         Telemetry schema and boundary checks
|   |-- safety_engine.py          Deterministic safety rules and fail-safe
|   |-- spoof_detection.py        Frozen / replay / rate-of-change detection
|   |-- maintenance_log_analyzer.py  Regex/dictionary log NLP
|   |-- predictive_model.py       StandardScaler + RandomForest pipeline
|   |-- train_model.py            Chronological training and evaluation
|   |-- decision_engine.py        Priority-based decision synthesis
|   |-- reporting.py              CSV, JSON and Markdown export
|   |-- generate_dataset.py       Dataset generator (CLI)
|   `-- main.py                   Orchestrator (--mode simulate | batch)
|-- data/                         Generated datasets
|-- models/predictive_model.joblib   Serialised model
|-- outputs/                      Reports and diagnostics
|-- logs/monitoring.log           Audit log
|-- tests/                        22 pytest tests
|-- docs/                         Architecture, logic, security, report
|-- README.md, requirements.txt, pytest.ini, .gitignore
```

CHAPTER 4
IMPLEMENTATION

This chapter describes how each component was implemented. The screenshots were captured from the running system on 15 October 2026.

4.1 Dataset Generation

The module generate_dataset.py creates all four datasets with a fixed random seed so that the results can be reproduced. The historical dataset contains 2,160 records covering 60 simulated time steps for each of the six joints of the six arms, with wear labels produced by empirical degradation equations. The current snapshot, the technician notes and the sensor-freezing scenario are generated in the same run. The command and its output are shown in Figure 4.1.

```
python -m src.generate_dataset
```

```
E:\Occ>python -m src.generate_dataset
2026-10-15 06:49:02,484 [INFO] generate_dataset: Generating historical training dataset (2,160 samples)...
2026-10-15 06:49:02,551 [INFO] generate_dataset: Saved historical dataset to E:\Occ\data\historical_joint_readings.csv (2160 rows)
2026-10-15 06:49:02,552 [INFO] generate_dataset: Generating current telemetry snapshot for 6 arms (36 rows)...
2026-10-15 06:49:02,554 [INFO] generate_dataset: Saved current telemetry to E:\Occ\data\current_telemetry.csv (36 rows)
2026-10-15 06:49:02,554 [INFO] generate_dataset: Generating free-text technician maintenance logs...
2026-10-15 06:49:02,555 [INFO] generate_dataset: Saved maintenance logs to E:\Occ\data\maintenance_logs.txt
2026-10-15 06:49:02,555 [INFO] generate_dataset: Generating spoofed sensor test scenario dataset...
2026-10-15 06:49:02,557 [INFO] generate_dataset: Saved spoofed sensor dataset to E:\Occ\data\spoofed_sensor_cases.csv (8 rows)

============================================================
 DATASET GENERATION COMPLETE
============================================================
 1. Historical Dataset:    E:\Occ\data\historical_joint_readings.csv (2160 rows)
 2. Current Telemetry:     E:\Occ\data\current_telemetry.csv (36 rows)
 3. Maintenance Logs:      E:\Occ\data\maintenance_logs.txt (12 lines)
 4. Spoofed Sensor Cases:  E:\Occ\data\spoofed_sensor_cases.csv (8 rows)
============================================================
```

Figure 4.1: Output of the dataset generator

4.2 Configuration Loader

All thresholds are kept in config/thresholds.json and are grouped into safety limits, joint-specific torque limits, predictive thresholds and cyber (integrity) thresholds. The module config_loader.py reads the file and validates its structure and value types. If a key is missing or a value is invalid, a ConfigurationError is raised and the system refuses to start. This prevents the pipeline from silently running with unsafe defaults.

4.3 Data Validator

The module data_validator.py checks the incoming telemetry before any analysis. It verifies that all required columns exist, that the values have numeric types and that they lie within physically possible boundaries. Records that cannot be interpreted are not discarded silently; they are passed on as invalid so that the safety engine can apply its fail-safe rule.

4.4 Deterministic Safety Engine

The safety engine compares the torque, temperature and vibration of each joint with its critical limits. A critical value immediately yields EMERGENCY_STOP together with a human-readable reason, for example the thermal event of arm_4 (88.5 degC against a limit of 85.0 degC) or the torque excursion of arm_6 (338.0 Nm against a limit of 320.0 Nm). Missing, NaN or infinite values are handled by a fail-safe branch that also requests a stop. The engine does not import or call the predictive model, and a dedicated test proves that its result is independent of the model.

4.5 Sensor Spoofing Detection

The module spoof_detection.py evaluates the sequence of readings of each joint and reports an integrity status of TRUSTED, SUSPICIOUS or CRITICAL_INTEGRITY_FAILURE. It implements the following checks:

- **Frozen sensor:** the standard deviation of a recent window is compared with a minimum value; a value of exactly 0.000000 indicates a stuck signal.
- **Replay:** duplicate or non-advancing timestamps indicate that old messages are being resent.
- **Rate of change:** a change between two consecutive samples that exceeds the physical limit is flagged as implausible.
- **Cross-sensor contradiction:** a calm reading on one sensor combined with extreme values on the others is treated as evidence of tampering.
- **Repetition:** repeated suspicious readings raise the status from a single warning to a failure.

In the eight-sample attack scenario the vibration signal of arm_5 stays frozen while the other sensors keep changing. The detector reports a standard deviation of 0.000000 together with a cross-sensor contradiction and marks the arm as CRITICAL_INTEGRITY_FAILURE.

4.6 Maintenance Log Analyzer

The module maintenance_log_analyzer.py reads the twelve technician notes, identifies the arm and joint mentioned in each note and classifies it with regular expressions and categorised dictionaries. Fault phrases such as gearbox backlash or abnormal noise accumulate into a concern score; phrases describing a verified replacement or repair credit the score; shift chatter without technical content is ignored. The resulting concern is a value between 0 and 1 for every arm. For example arm_2 receives 0.45 because of an active note on the elbow, arm_3 receives 0.55 because of severe backlash on the shoulder, and arm_1 receives 0.00 because an earlier concern was cleared by a documented repair.

4.7 Predictive Wear Model

The module predictive_model.py defines a scikit-learn pipeline consisting of a StandardScaler and a RandomForestClassifier with 100 trees and a maximum depth of 8. Class weights are set to balanced because only 2.8% of the samples are failures. The script train_model.py loads the 2,160 historical records, sorts them in time, uses the first 80% (1,728 samples) for training and the last 20% (432 samples) for testing, and writes the metrics to outputs/model_evaluation.json and the pipeline to models/predictive_model.joblib. The run is shown in Figure 4.2.

```
python -m src.train_model
```

```
E:\Occ>python -m src.train_model
2026-10-15 06:49:10,346 [INFO] train_model: Loaded 2160 records from E:\Occ\data\historical_joint_readings.csv
2026-10-15 06:49:10,352 [INFO] train_model: Training PredictiveModel on 1728 training samples...
2026-10-15 06:49:10,545 [INFO] train_model: Saved evaluation metrics to E:\Occ\outputs\model_evaluation.json
2026-10-15 06:49:10,559 [INFO] predictive_model: Saved predictive model pipeline to E:\Occ\models\predictive_model.joblib

============================================================
 PREDICTIVE MAINTENANCE MODEL TRAINING & EVALUATION REPORT
============================================================
 Train Samples: 1728 | Test Samples: 432
 Positive Failure Rate in Test: 2.8%
------------------------------------------------------------
 Accuracy:  0.9884
 Precision: 0.7333
 Recall:    0.9167
 F1-Score:  0.8148
 ROC-AUC:   0.9970
------------------------------------------------------------
 Confusion Matrix: [TN=416, FP=4, FN=1, TP=11]
============================================================
```

Figure 4.2: Model training and evaluation report

Table 4.1: Evaluation of the predictive model on the 432-sample hold-out set

Metric	Value	Interpretation
Accuracy	0.9884	Overall share of correct classifications
Precision	0.7333	11 of 15 predicted failures were real, which limits unnecessary stoppages
Recall	0.9167	11 of 12 impending failures were detected
F1-score	0.8148	Balance between precision and recall
ROC-AUC	0.9970	Very good ranking of failing versus healthy joints

Table 4.2: Confusion matrix of the hold-out set

	Predicted healthy	**Predicted failure**
Actually healthy	TN = 416	FP = 4
Actually failing	FN = 1	TP = 11

4.8 Decision Engine

The module decision_engine.py receives the joint-level findings of the four channels and produces exactly one decision per arm. It records the critical joint, the peak 24-hour risk, whether an emergency stop was triggered, the sensor integrity, the maintenance concern and the primary triggering reason. The precedence is EMERGENCY_STOP, SCHEDULE_MAINTENANCE, MONITOR and SAFE. Alerts from several joints of the same arm are aggregated, so that a single critical joint determines the decision of the whole arm. A failed integrity check or a safety stop cannot be reversed by any lower-priority evidence.

4.9 Reporting and Command-Line Interface

The module reporting.py exports the arm summary (CSV and JSON), the per-joint diagnostics, the spoof detection report, the maintenance analysis, the model evaluation and a final Markdown report. The orchestrator main.py offers two modes: simulate, which runs the complete demonstration, and batch, which analyses a given telemetry CSV and log file. Every run writes an audit record to logs/monitoring.log.

```
python -m src.main --mode simulate
python -m src.main --mode batch ^
    --input data/current_telemetry.csv ^
    --logs data/maintenance_logs.txt
```

CHAPTER 5
TESTING AND RESULTS

5.1 Test Environment

All tests were run on a Windows computer with Python 3.13.14, pytest 8.3.3 and pluggy 1.6.0 from the project folder E:\Occ. The test configuration is stored in pytest.ini and tests/conftest.py.

5.2 Automated Test Suite

The suite contains 22 tests in five modules. Their coverage is summarised in Table 5.1, and the complete verbose output is shown in Figure 5.1. All 22 tests passed in 1.33 seconds.

Table 5.1: Summary of the automated tests

Test module	Tests	Behaviour verified	Result
test_safety_engine.py	6	Normal telemetry is safe; critical torque, temperature and vibration give EMERGENCY_STOP; missing and invalid data fail safe; the safety check is independent of the model	Pass
test_spoof_detection.py	4	Repeated suspicious readings raise a warning; duplicate replayed timestamps are detected; a spoofed low reading with cross-sensor contradiction is detected; implausible rate of change is flagged	Pass
test_maintenance_log_analyzer.py	3	Unresolved relevant notes raise concern; irrelevant notes do not trigger maintenance; a documented repair clears the concern	Pass
test_decision_engine.py	6	Normal telemetry gives SAFE; a stop cannot be overridden by low risk; moderate risk gives MONITOR; high risk gives SCHEDULE_MAINTENANCE; exactly one valid decision; multi-joint alert aggregation	Pass
test_pipeline.py	3	Invalid configuration is rejected; dataset generation is reproducible; exports are valid CSV and JSON	Pass

```
E:\Occ>pytest -v
============================================= test session starts =============================================
platform win32 -- Python 3.13.14, pytest-8.3.3, pluggy-1.6.0 -- C:\Users\vklak\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: E:\Occ
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.14.2, asyncio-0.24.0
asyncio: mode=Mode.STRICT, default_loop_scope=function
collected 22 items

tests/test_decision_engine.py::test_normal_telemetry_produces_safe PASSED                                [  4%]
tests/test_decision_engine.py::test_emergency_stop_cannot_be_overridden_by_low_risk PASSED               [  9%]
tests/test_decision_engine.py::test_moderate_predictive_risk_produces_monitor PASSED                     [ 13%]
tests/test_decision_engine.py::test_high_predictive_risk_produces_schedule_maintenance PASSED            [ 18%]
tests/test_decision_engine.py::test_exactly_one_valid_final_decision PASSED                              [ 22%]
tests/test_decision_engine.py::test_multiple_joint_level_alerts_aggregation PASSED                       [ 27%]
tests/test_maintenance_log_analyzer.py::test_relevant_unresolved_notes_increase_concern PASSED           [ 31%]
tests/test_maintenance_log_analyzer.py::test_irrelevant_notes_do_not_trigger_maintenance PASSED          [ 36%]
tests/test_maintenance_log_analyzer.py::test_documented_repair_clears_concern PASSED                     [ 40%]
tests/test_pipeline.py::test_invalid_config_rejected_with_error PASSED                                   [ 45%]
tests/test_pipeline.py::test_dataset_generation_reproducibility PASSED                                   [ 50%]
tests/test_pipeline.py::test_pipeline_exports_valid_csv_and_json PASSED                                  [ 54%]
tests/test_safety_engine.py::test_normal_telemetry_is_safe PASSED                                        [ 59%]
tests/test_safety_engine.py::test_critical_torque_triggers_emergency_stop PASSED                         [ 63%]
tests/test_safety_engine.py::test_critical_temperature_triggers_emergency_stop PASSED                    [ 68%]
tests/test_safety_engine.py::test_critical_vibration_triggers_emergency_stop PASSED                      [ 72%]
tests/test_safety_engine.py::test_missing_and_invalid_data_failsafe PASSED                               [ 77%]
tests/test_safety_engine.py::test_safety_check_independent_of_model PASSED                               [ 81%]
tests/test_spoof_detection.py::test_repeated_suspicious_readings_trigger_warning PASSED                  [ 86%]
tests/test_spoof_detection.py::test_duplicate_replayed_timestamps_detected PASSED                        [ 90%]
tests/test_spoof_detection.py::test_spoofed_low_reading_with_cross_sensor_contradiction PASSED           [ 95%]
tests/test_spoof_detection.py::test_implausible_rate_of_change PASSED                                    [100%]

============================================== 22 passed in 1.33s ==============================================
```

Figure 5.1: Output of pytest -v: 22 tests passed

5.3 Operational Results

The command python -m src.main --mode simulate analyses the six arms and prints the operational summary shown in Figure 5.2. All four permitted decisions occur, which demonstrates the complete decision hierarchy.

```
================================================================================================
 ROBOTIC ARM JOINT HEALTH MONITORING SYSTEM - OPERATIONAL SUMMARY
================================================================================================
ARM ID   | FINAL DECISION         | CRITICAL JOINT  | MAX RISK   | INTEGRITY
------------------------------------------------------------------------------------------------
arm_1    | SAFE                   | joint_1         | 0.0%       | TRUSTED
arm_2    | MONITOR                | joint_3         | 45.1%      | TRUSTED
arm_3    | SCHEDULE_MAINTENANCE   | joint_2         | 99.0%      | TRUSTED
arm_4    | EMERGENCY_STOP         | joint_2         | 1.0%       | TRUSTED
arm_5    | EMERGENCY_STOP         | joint_2         | 1.0%       | CRITICAL_INTEGRITY_FAILURE
arm_6    | EMERGENCY_STOP         | joint_2         | 1.9%       | TRUSTED
================================================================================================
Precedence: EMERGENCY_STOP > SCHEDULE_MAINTENANCE > MONITOR > SAFE
================================================================================================
```

Figure 5.2: Operational summary produced in simulation mode

Table 5.2: Final operational decision for each robotic arm

Arm	Decision	Joint	Risk	Integrity	Concern	Primary triggering reason
arm_1	SAFE	joint_1	0.0%	TRUSTED	0.00	All joints nominal; a past repair cleared an earlier concern
arm_2	MONITOR	joint_3	45.1%	TRUSTED	0.45	Moderate vibration wear (3.50 mm/s) and an active technician note on the elbow
arm_3	SCHEDULE_ MAINTENANCE	joint_2	99.0%	TRUSTED	0.55	High predictive risk (99.0%) and a severe gearbox backlash note on the shoulder
arm_4	EMERGENCY_ STOP	joint_2	1.0%	TRUSTED	0.00	Thermal limit crossed: 88.5 degC against the 85.0 degC limit
arm_5	EMERGENCY_ STOP	joint_2	1.0%	CRITICAL_ INTEGRITY_ FAILURE	0.00	Frozen sensor (standard deviation 0.000000) with cross-sensor contradiction
arm_6	EMERGENCY_ STOP	joint_2	1.9%	TRUSTED	0.00	Torque limit crossed: 338.0 Nm against the 320.0 Nm limit; low 1.9% ML risk overridden

Risk is the peak 24-hour failure probability of the critical joint. Arms arm_4, arm_5 and arm_6 triggered an emergency stop; arm_1, arm_2 and arm_3 did not.

5.4 Discussion

Normal and borderline arms. arm_1 is SAFE with a risk of 0.0%. Its maintenance notes contain a documented repair, which shows that the log analyser does not keep alarming about a problem that has been fixed. arm_2 is classified as MONITOR because it combines moderate vibration with an active technician note, neither of which alone justifies a stop.

Predictive maintenance. arm_3 has a peak 24-hour risk of 99.0% and a severe gearbox backlash note. Neither the model nor the note is a measured safety violation, so the arm receives SCHEDULE_MAINTENANCE rather than a stop. This is the intended use of the predictive channel: early warning that allows planned repair.

Safety overrides prediction. arm_4 and arm_6 show the value of the deterministic gate. In both cases the model gave a very low risk (1.0% and 1.9%), yet a measured limit was exceeded and the arm was stopped. The arm_6 result is a direct demonstration that a low model risk cannot override a safety stop.

Cyber integrity. arm_5 has a normal-looking, constant vibration value that the model rated as only 1.0% risky. The integrity channel identified the frozen signal and the cross-sensor contradiction and forced an EMERGENCY_STOP. Without the anti-spoofing module a successful sensor-freezing attack would have hidden the real

condition of the joint.

Model quality. The model found 11 of the 12 impending failures with only four false alarms in 432 samples. The single missed failure illustrates why the model is used for planning only and not as a protection mechanism.

5.5 Limitations

- Telemetry, failure labels and technician notes are synthetic; real deployment requires calibration against measured data.
- The system is a decision-support tool and not a certified safety controller (ISO 13849 PL e, IEC 61508 SIL 3).
- The integrity detector handles frozen sensors, replays and rate-of-change anomalies, but a sophisticated coordinated replay synchronised with the real dynamics would require cryptographic attestation.
- The log analyser is rule-based and recognises only the phrases contained in its dictionaries.
- The test set contains only 12 failures, so the quoted precision and recall have wide confidence intervals.

CHAPTER 6
SECURITY CONSIDERATIONS

6.1 Threat Model

The system assumes that an attacker or a fault may manipulate the telemetry that reaches the monitor, but cannot change the code or the configuration file. Table 6.1 lists the considered threats and the countermeasures that are implemented.

Table 6.1: Threats, detection methods and system responses

Threat	Detection method	Response
Frozen or stuck sensor	Zero standard deviation of a recent window	Integrity failure and EMERGENCY_STOP
Replay of old messages	Duplicate or non-advancing timestamps	Integrity warning or failure
Injected step jump	Rate-of-change plausibility limit	Reading flagged as implausible
Falsified single sensor	Cross-sensor contradiction rule	Integrity failure
Corrupt or missing data	Schema validation and NaN/Inf check	Fail-safe stop
Wrong configuration	Strict schema validation	ConfigurationError, system does not start
Coordinated multi-sensor replay	Not detectable by statistics alone	Requires signed packets (TPM/HSM)

6.2 Design Trade-offs

Strict thresholds and a fail-safe attitude increase the chance of a false stop, which costs production time, but a missed stop can cost safety. The system therefore stops whenever data cannot be trusted and relies on the repetition rule to separate single glitches from persistent manipulation. The predictive model is tuned for high recall, accepting some false alarms, because it only schedules maintenance and does not itself stop the machine.

6.3 Safety Disclaimer

The pipeline is a demonstration and diagnostic decision-support system. Actual industrial machinery requires independently certified hardware safety relays, dual-channel interlocks and a formal safety assessment. The software emergency stop produced by this project should be treated as a signal to such a certified controller and never as the only protective measure.

CHAPTER 7
FUTURE ENHANCEMENTS

The modular design allows several extensions without redesigning the system:

- **Real data:** connect the pipeline to live joint telemetry from a robot controller or PLC and recalibrate the thresholds and the model.
- **Cryptographic attestation:** sign sensor packets with hardware keys to detect coordinated replay attacks.
- **Richer models:** add remaining-useful-life regression, anomaly detection and drift monitoring; compare gradient boosting and sequence models.
- **Better text understanding:** extend the dictionaries or add a local language model for the technician notes.
- **Real-time operation:** stream the data through MQTT or OPC UA and publish decisions to a dashboard.
- **Notifications:** send e-mail, SMS or mobile alerts for stops and maintenance requests.
- **Certified safety path:** forward the stop signal to a safety-rated controller for actual actuation.
- **Scalability:** package the system in containers and monitor many robots across several sites.

CHAPTER 8
CONCLUSION

This project designed, implemented and tested a modular Robotic Arm Joint Health Monitoring System that reduces four independent evidence channels to one operational decision per arm. The deterministic safety engine guarantees that measured limit violations and invalid data always lead to an emergency stop, the anti-spoofing module protects the decision process from frozen, replayed and implausible sensor data, the log analyser makes use of technician knowledge without any external service, and the RandomForest model provides a 24-hour failure risk with 98.84% accuracy, 91.67% recall and 0.997 ROC-AUC on a chronological hold-out set.

The simulation demonstrated all four decisions across six arms and showed in particular that a low predictive risk cannot override a safety stop and that a successful sensor-freezing attack is caught by the integrity channel. All 22 automated tests passed, and the configuration-driven, reproducible design makes the system easy to inspect and extend. With real sensor data, hardware-backed authentication and a certified safety controller, the same architecture can form the basis of a production condition-monitoring solution for industrial robots.

REFERENCES

[1] ISO 13849-1, "Safety of machinery - Safety-related parts of control systems - Part 1: General principles for design," International Organization for Standardization.

[2] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] A. K. S. Jardine, D. Lin and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," Mechanical Systems and Signal Processing, vol. 20, no. 7, pp. 1483-1510, 2006.

[5] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[6] IEC 61508, "Functional safety of electrical/electronic/programmable electronic safety-related systems," International Electrotechnical Commission.

[7] scikit-learn developers, "scikit-learn documentation." [Online]. Available: https://scikit-learn.org/stable/

[8] pytest development team, "pytest documentation." [Online]. Available: https://docs.pytest.org/

APPENDIX A: OPERATIONAL COMMANDS

```
# Generate the datasets
python -m src.generate_dataset

# Train and evaluate the predictive model
python -m src.train_model

# Run the six-arm simulation demonstration
python -m src.main --mode simulate

# Analyse a telemetry CSV together with maintenance logs
python -m src.main --mode batch ^
    --input data/current_telemetry.csv ^
    --logs data/maintenance_logs.txt

# Run the automated test suite
pytest -v
```

APPENDIX B: OUTPUT ARTEFACTS

Table B.1: Files written by the pipeline

Artefact	Location	Content
Arm decision summary (CSV)	outputs/arm_health_summary.csv	One row per arm with final decision
Decision hierarchy (JSON)	outputs/arm_health_summary.json	Complete decision and diagnostic hierarchy
Joint diagnostics (CSV)	outputs/joint_diagnostics.csv	Per-joint telemetry diagnostics
Spoof detection report (CSV)	outputs/spoof_detection_report.csv	Integrity findings per joint
Maintenance analysis (JSON)	outputs/maintenance_analysis.json	Parsed notes and concern scores
Model evaluation (JSON)	outputs/model_evaluation.json	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix
Final report (Markdown)	outputs/final_report.md	Consolidated operational report
Audit log	logs/monitoring.log	Execution record
Model artefact	models/predictive_model.joblib	Serialised scikit-learn pipeline

APPENDIX C: SOURCE FILES AND TESTS

Table C.1: Source modules

File	Responsibility
src/config_loader.py	Loads thresholds.json and validates the schema (ConfigurationError)
src/data_validator.py	Telemetry schema and physical-boundary validation
src/safety_engine.py	Deterministic immediate safety engine and fail-safe handler
src/spoof_detection.py	Frozen-sensor, replay and rate-of-change detection
src/maintenance_log_analyzer.py	Zero-API regex and dictionary log analysis
src/predictive_model.py	StandardScaler + RandomForestClassifier pipeline
src/train_model.py	Chronological training and evaluation
src/decision_engine.py	Multi-joint priority decision synthesis
src/reporting.py	CSV, JSON and Markdown exporter
src/generate_dataset.py	Dataset generation CLI
src/main.py	CLI orchestrator (simulate and batch modes)

Table C.2: Documentation delivered with the project

File	Content
README.md	User guide, installation and commands
docs/architecture.md	Subsystems and data-flow diagrams
docs/decision_logic.md	Priority hierarchy, thresholds and worked case studies
docs/security_analysis.md	Threat model, attack vectors and trade-offs
docs/report.md	Engineering report with executed results